Binary Search

B.Tech DAA Lab Presentation

Student Name :

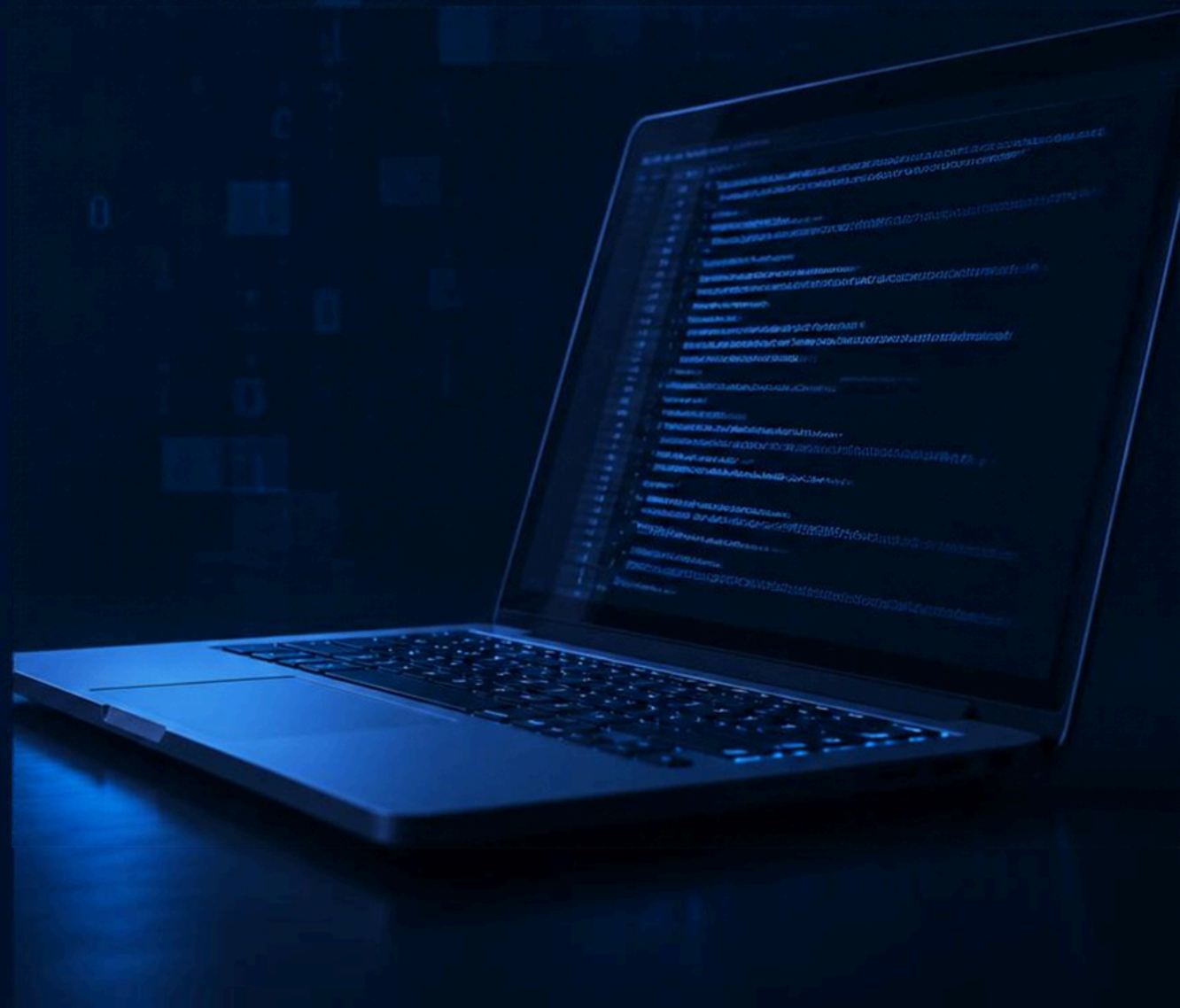
Roll Number :

Class :

Faculty Name :

19 October 2026

Generated using Canva AI



Introduction

Binary Search

An efficient searching algorithm designed for sorted arrays.

It quickly locates the position of a target value.

Widely used in computer science and software development.



Problem Statement

Find a target value in a **sorted array.**

Linear search scans every element, taking O(n) time and becomes inefficient at scale.

Goal: return the index of the target if it exists, or report it as not found, using **fewer comparisons.**

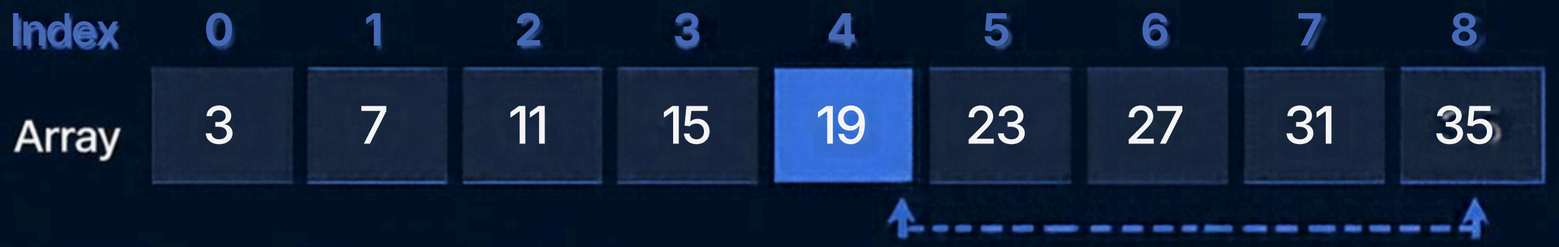


Working Principle
with Sorted Array Example

Finding target value 23 in a sorted array.

01

Middle element is 19 at index 4. Since 23 > 19, search the **right half.**



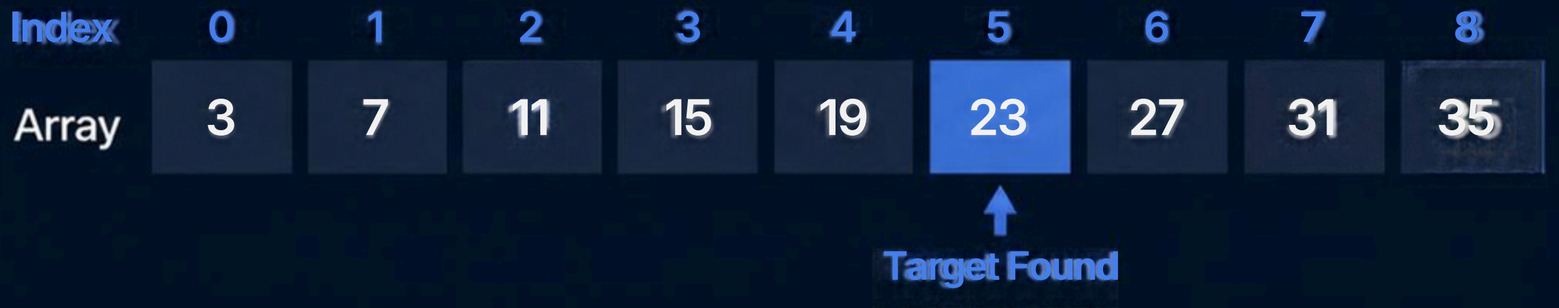
02

New middle element is 27 at index 6. Since 23 < 27, search the **left half.**



03

New middle element is 23 at index 5. Target found. **Search complete.**



Step-by-Step Algorithm

A simple procedure to find a target in a sorted array.

01 **Initialize the search range.**
Set low = 0 and high = n - 1.

↓

02 **Find the middle position.**
Compute mid = (low + high) / 2.

↓

03 **Compare the middle element.**
If equal, return mid.

↓

04 **Adjust the search range.**
If the middle value is smaller, search right half. If it is larger, search left half.

↓

05 **Not found.**
Return -1.

Pseudocode

Binary Search (Iterative)

Find the position of
a target in a sorted array.
Return its index if found,
otherwise return **-1**.

```
procedure BINARY SEARCH(arr, target)
    low < 0                                  // start at the first index
    high < length(arr) - 1                   // start at the last index
    while low < high                         // repeat while range is valid
        mid < (low + high) // 2              // find middle index
        if arr[mid] = target                 // target found
            return mid
        else if arr[mid] < target            // target is in right half
            low < mid + 1                    // discard left half
        else                                 // target is in left half
            high < mid - 1                   // discard right half
    return -1
```

Time and Space Complexity

Best-case time complexity

O(1)

Average and worst-case time complexity

O(log n)

Space complexity (iterative)

O(1)



Advantages, Limitations, and Applications

01 Advantages

Fast search on sorted data

Simple to implement

Low memory usage

02 Limitations

Requires sorted array

Not suitable for linked lists without indexing

03 Applications

Searching in databases

Libraries

Coding competitions

System utilities

Conclusion and References

Binary Search is an efficient searching algorithm with **logarithmetic time complexity**.

It requires **sorted data** to work correctly.

It is **widely used** in computer science and software development.



References

Introduction to Algorithms by Cormen, Leiserson, Rivest, and Stein

Online tutorials and documentation

Thank you!

Questions are welcome.